

# NCERT AI TUTOR

RAG-Powered Multilingual Study Assistant

Complete Build Plan — Ayush Halpati · June 2026

**PURPOSE:** This is your complete guide to building the NCERT AI Tutor — a RAG-based GenAI project that closes your skill gap, gives you a live deployed product, and makes you a credible GenAI Engineer applicant. Read everything before writing a single line of code.

## 1. WHAT ARE WE BUILDING?

### Project Name: NCERT AI Tutor

A Retrieval-Augmented Generation (RAG) chatbot that answers student questions in English and Hindi using NCERT textbook content as its knowledge base. Students type a question, the system finds the relevant NCERT chapter content, and an LLM generates a clear, accurate answer grounded in that content — not hallucinated from training data.

### One-line pitch for interviews:

"I built an RAG pipeline that ingests NCERT PDFs into a ChromaDB vector store, uses LangChain to retrieve relevant chunks, and generates answers via Groq API — deployed on Streamlit with Hindi/English prompt switching."

### Why this project specifically:

- Uses LangChain, RAG, and vector databases — the exact 3 skills missing from your GenAI profile
- NCERT content is free, public, legally downloadable — zero data sourcing problem
- India-specific — every Indian recruiter immediately understands it
- Deployable on Streamlit Cloud for free — live URL on resume same day you finish
- Completable in 5–7 days with AI agents — not 6 months

## 2. SKILLS YOU MUST UNDERSTAND BEFORE CODING

**WARNING:** Do NOT start coding until you understand these 6 concepts. You will get stuck and your AI agents will give wrong answers if you don't know what you're building. Read each concept, watch one short video, then proceed.

| Skill / Concept            | What It Is                                                                                             | Why You Need It                                                                                 | Time to Learn |
|----------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------|
| Embeddings                 | Converting text into numbers (vectors) that capture meaning. Similar sentences get similar vectors.    | NCERT documents are stored as embeddings. Without this you can't understand why ChromaDB works. | 1 hour        |
| Vector Database (ChromaDB) | A database that stores embeddings and finds the most similar ones fast using cosine similarity.        | This is how the system finds the right NCERT paragraph for each student question.               | 2 hours       |
| RAG Pipeline               | Retrieve → Augment → Generate. Find relevant documents first, then give them to the LLM as context.    | Core architecture of the entire project. Understand this before touching any code.              | 2 hours       |
| LangChain                  | Python framework that chains: document loading → text splitting → embedding → retrieval → LLM call.    | You use LangChain to build the entire pipeline in ~50 lines of code.                            | 3 hours       |
| Groq API                   | Free API that runs open-source LLMs (Llama 3, Mixtral) at very high speed. You already use it in ZENI. | Used as the generation step of the RAG pipeline. Replaces OpenAI API, completely free.          | 30 min        |
| Prompt Templates           | Structured prompts with variable slots. LangChain injects retrieved context + question into the LLM.   | Controls answer quality. Hindi/English language switching happens entirely here.                | 1 hour        |

#### Learning resources (free, fast):

- **Embeddings:** Search 'What are embeddings' on YouTube — watch any 15-min explanation
- **RAG concept:** [python.langchain.com/docs/tutorials/rag](https://python.langchain.com/docs/tutorials/rag) — read the overview, don't code yet
- **ChromaDB:** [docs.trychroma.com](https://docs.trychroma.com) — Getting Started page only
- **LangChain RAG:** [python.langchain.com/docs/tutorials/rag](https://python.langchain.com/docs/tutorials/rag) — full tutorial, follow step by step
- **Groq:** [console.groq.com](https://console.groq.com) — sign up, get API key, run one test call in Python

### 3. TECH STACK — WHAT YOU INSTALL AND WHY

|                              |                                                                             |
|------------------------------|-----------------------------------------------------------------------------|
| <b>Python 3.10+</b>          | Core language — you already know this.                                      |
| <b>langchain</b>             | The RAG framework that chains everything together.                          |
| <b>langchain-community</b>   | Extra LangChain loaders including PyPDFLoader for reading PDFs.             |
| <b>langchain-groq</b>        | Connects LangChain to Groq API for LLM generation step.                     |
| <b>chromadb</b>              | Local vector database — runs on your laptop, no cloud account needed.       |
| <b>pypdf</b>                 | Reads and extracts raw text from NCERT PDF files.                           |
| <b>sentence-transformers</b> | Creates embeddings using HuggingFace all-MiniLM-L6-v2 model — runs locally. |
| <b>streamlit</b>             | Builds the web UI and deploys for free on Streamlit Cloud.                  |
| <b>python-dotenv</b>         | Manages your Groq API key securely in a .env file — never hardcode keys.    |

#### Full install command — paste this into terminal:

```
pip install langchain langchain-community langchain-groq chromadb pypdf streamlit sentence-transformers python-dotenv
```

## 4. SYSTEM ARCHITECTURE — HOW IT WORKS END TO END

|                   |                                                                                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Step 1 — Load     | Run ingest.py once. PyPDFLoader reads all NCERT PDFs in your data/ncert_pdfs/ folder and extracts raw text page by page.                                                |
| Step 2 — Split    | RecursiveCharacterTextSplitter breaks the text into overlapping chunks of 500 characters with 50-character overlap. Overlap ensures no sentence is cut at the boundary. |
| Step 3 — Embed    | Each chunk is converted into a vector of 384 numbers using HuggingFace all-MiniLM-L6-v2. This model runs fully offline on your laptop — no API call needed.             |
| Step 4 — Store    | All vectors are saved to ChromaDB on disk in ./chroma_db/. This happens once. Next run, ChromaDB loads from disk — no re-embedding.                                     |
| Step 5 — Query    | Student types a question. That question is also embedded into a vector. ChromaDB finds the 4 most similar NCERT chunks using cosine similarity.                         |
| Step 6 — Augment  | LangChain builds a prompt combining: [Retrieved NCERT chunks as context] + [Student question] + [Language instruction: respond in Hindi or English].                    |
| Step 7 — Generate | Groq API (Llama 3 model) receives the full prompt and generates a grounded answer. Response is streamed live to the Streamlit UI character by character.                |

## 5. PROJECT FOLDER STRUCTURE

```
ncert-ai-tutor/ ■■■ app.py <- Streamlit UI (run this to start the app) ■■■ rag_pipeline.py
<- LangChain RAG logic (retriever + LLM chain) ■■■ ingest.py <- Load PDFs, create
embeddings, store in ChromaDB ■■■ data/ ■■■ ncert_pdfs/ <- Put your downloaded NCERT PDF
files here ■■■ chroma_db/ <- ChromaDB vector store (auto-created by ingest.py) ■■■ .env <-
GROQ_API_KEY=your_key_here ■■■ .gitignore <- Must include: .env, chroma_db/,
data/ncert_pdfs/ ■■■ requirements.txt <- All pip dependencies ■■■ README.md <- Project
description, live URL, how to run
```

## 6. DAY-BY-DAY BUILD PLAN

STRATEGY: Use Claude or ChatGPT as your coding agent. Give it ONE phase at a time. Do NOT paste the entire plan into the agent. Complete each phase, test it manually, then move on.

| Day / Phase  | What You Build                                                                             | How to Test                                                                               | AI Agent Prompt to Use                                                                                                                                                                 |
|--------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 1 Setup  | Install all libraries. Get Groq API key from console.groq.com. Download 3 NCERT PDFs.      | Run: python -c 'import langchain, chromadb, groq' — should not error.                     | Ask Claude: Set up Python project for LangChain RAG with ChromaDB and Groq. Show folder structure and requirements.txt                                                                 |
| Day 2 Ingest | Write ingest.py — load PDFs, split text into chunks, create embeddings, store in ChromaDB. | Run ingest.py — should print number of chunks stored. Check chroma_db/ folder is created. | Ask Claude: Write ingest.py using LangChain PyPDFLoader, RecursiveCharacterTextSplitter chunk_size=500, HuggingFaceEmbeddings all-MiniLM-L6-v2, ChromaDB persist_directory=./chroma_db |

| Day / Phase        | What You Build                                                                                       | How to Test                                                                                               | AI Agent Prompt to Use                                                                                                                                                               |
|--------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 3 RAG Core     | Write rag_pipeline.py — load ChromaDB, create retriever, build LangChain RAG chain with Groq.        | Call get_answer('What is photosynthesis?', 'English') in Python shell — should return NCERT-based answer. | Ask Claude: Write rag_pipeline.py with LangChain retriever from ChromaDB, ChatGroq llama3-8b-8192, PromptTemplate with context and question variables, return answer and source docs |
| Day 4 Streamlit UI | Write app.py — question text input, Hindi/English radio toggle, display streamed answer and sources. | Run streamlit run app.py — app opens in browser. Ask one question in both languages.                      | Ask Claude: Write Streamlit app.py that imports get_answer from rag_pipeline.py, shows language radio button, streams LLM response, shows source document names below answer         |
| Day 5 Test + Fix   | Ask 20 real NCERT questions. Note where retrieval fails. Adjust chunk size or prompt template.       | 17 out of 20 questions should retrieve correct NCERT chapter. Fix the ones that fail.                     | You do this manually — AI agents cannot test for you. This is the most important day.                                                                                                |
| Day 6 Deploy       | Push cleaned code to GitHub. Deploy on Streamlit Cloud. Get live URL.                                | Share the live URL with someone and ask them to test 5 questions.                                         | Ask Claude: Write README.md for NCERT AI Tutor project with project description, tech stack, how to run locally, and deployment instructions                                         |
| Day 7 Optional     | Add conversation memory, more NCERT subjects, better UI, source page numbers in citations.           | Optional — only if you have time before interview calls start.                                            | Ask Claude: Add ConversationBufferMemory to my LangChain RAG chain so it remembers previous questions                                                                                |

## 7. EXACT PROMPTS TO USE WITH YOUR AI AGENTS

Use these word-for-word with Claude / ChatGPT / Antigravity:

### Prompt 1 — Project Setup:

I am building an NCERT RAG chatbot in Python. Tech stack: LangChain, ChromaDB, Groq API (model: llama3-8b-8192), sentence-transformers (all-MiniLM-L6-v2), Streamlit. Create the folder structure, empty skeleton files (app.py, rag\_pipeline.py, ingest.py), and requirements.txt. Do not write any logic yet.

### Prompt 2 — Ingest Pipeline (Day 2):

Write ingest.py for my NCERT RAG project. It must: 1) Load all PDFs from data/ncert\_pdfs/ using LangChain PyPDFLoader, 2) Split text using RecursiveCharacterTextSplitter with chunk\_size=500 chunk\_overlap=50, 3) Create embeddings using HuggingFaceEmbeddings model=all-MiniLM-L6-v2, 4) Store vectors in ChromaDB at persist\_directory=./chroma\_db, 5) Print total number of chunks stored at the end.

### Prompt 3 — RAG Pipeline (Day 3):

Write rag\_pipeline.py for my NCERT RAG project. It must: 1) Load ChromaDB from ./chroma\_db using HuggingFaceEmbeddings all-MiniLM-L6-v2, 2) Create retriever with k=4 (top 4 chunks), 3) Create PromptTemplate with variables: context, question, language, 4) Use ChatGroq with model llama3-8b-8192 and GROQ\_API\_KEY from .env, 5) Build a RetrievalQA or LCEL chain, 6) Expose function: get\_answer(question: str, language: str) -> dict with keys answer and sources.

### Prompt 4 — Streamlit UI (Day 4):

Write app.py for my NCERT AI Tutor using Streamlit. It must: 1) Show title: NCERT AI Tutor, 2) Show radio buttons: Language: English / Hindi, 3) Show text input: Ask your question, 4) On submit call get\_answer(question, language) from rag\_pipeline.py, 5) Display the answer in a styled box, 6) Display source document names below the answer under heading: Sources Used.

#### Prompt 5 — Debug (when stuck):

I am building an NCERT RAG chatbot. Stack: Python, LangChain, ChromaDB, Groq, Streamlit. I am getting this error: [PASTE YOUR FULL ERROR HERE]. Here is the relevant code: [PASTE THE FILE THAT HAS THE ERROR]. Fix the error, explain what caused it, and show the corrected code.

## 8. WHERE TO GET NCERT PDFs — FREE AND LEGAL

Download directly from [ncert.nic.in](https://ncert.nic.in) — official Government of India website. All content is public domain. Recommended PDFs to start with:

- Class 10 Science — good variety of topics, clear language, strong for testing
- Class 10 Mathematics — structured content, great for testing retrieval accuracy
- Class 12 Physics Part 1 — more complex, shows the system handles advanced content

*START WITH 2 PDFs ONLY. Once the pipeline works correctly, add more. Do not load 20 PDFs on Day 1 — ChromaDB will take time to embed everything and you won't know if the pipeline is working.*

## 9. HOW TO IMPLEMENT HINDI — IT IS SIMPLER THAN YOU THINK

You do NOT need to fine-tune any model for Hindi. Groq's Llama 3 already speaks Hindi fluently. The entire Hindi implementation is a single line in your PromptTemplate:

English prompt instruction: 'You are a helpful NCERT tutor. Answer ONLY using the context below. Answer in clear English.' Hindi prompt instruction: 'Aap ek helpful NCERT tutor hain. Sirf neeche diye gaye context ka use karke jawab dein. Hindi mein jawab dein.'

In your Streamlit app, pass language as a parameter to get\_answer(). The PromptTemplate selects the correct instruction string based on that parameter. That is the entire implementation — no translation API, no extra model, zero extra cost.

## 10. GITHUB AND STREAMLIT CLOUD DEPLOYMENT

#### Step 1 — GitHub checklist before pushing:

- Create .gitignore file — add: .env, chroma\_db/, data/ncert\_pdfs/
- Never push .env — your Groq API key will be exposed and invalid immediately
- Never push chroma\_db/ — too large, rebuild on deployment using preloaded data
- Write README.md with: what it does, tech stack, live demo URL, how to run locally

#### Step 2 — Streamlit Cloud deployment (completely free):

|        |                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------|
| Step 1 | Push your code to a public GitHub repository                                                         |
| Step 2 | Go to <a href="https://share.streamlit.io">share.streamlit.io</a> — sign in with your GitHub account |
| Step 3 | Click 'New app' — select your repository and set app.py as the main file                             |
| Step 4 | Click 'Advanced settings' — add secret: GROQ_API_KEY = paste_your_key_here                           |
| Step 5 | For ChromaDB: commit a pre-built small chroma_db/ with 2 PDFs so it works on cloud                   |

|        |                                                          |
|--------|----------------------------------------------------------|
| Step 6 | Click Deploy — your live URL is ready in under 2 minutes |
|--------|----------------------------------------------------------|

## 11. HOW TO EXPLAIN THIS PROJECT IN INTERVIEWS

When asked 'Tell me about your GenAI project' — use this exact structure: Problem → Architecture → What you built → What you measured → What you learned. Practice until you can say this in under 2 minutes without looking at notes.

|                 |                                                                                                                                                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem         | NCERT students struggle to find specific answers quickly from dense textbook chapters. I built an AI tutor that gives grounded, accurate answers from actual NCERT content — not hallucinated responses.                |
| Architecture    | RAG pipeline: PyPDFLoader ingests NCERT PDFs, LangChain splits and embeds text into ChromaDB using sentence-transformers all-MiniLM-L6-v2, Groq API with Llama 3 generates answers with retrieved context as grounding. |
| What I Built    | ingest.py for one-time vector store creation, rag_pipeline.py for the full LangChain RAG chain, Streamlit UI with Hindi/English language toggle, deployed live on Streamlit Cloud.                                      |
| What I Measured | Tested 20 real NCERT questions — correct context retrieved in top-4 chunks for 17 out of 20. Hindi responses generated correctly without any translation API.                                                           |
| What I Learned  | How RAG fundamentally works. Why chunk size and overlap matter for retrieval quality. How to ground LLM outputs in verified content to prevent hallucination. How sentence-transformers create semantic similarity.     |

**Common follow-up questions — memorize these answers:**

- Why RAG instead of fine-tuning?:** Fine-tuning is expensive and makes the model memorize data. RAG keeps knowledge external and updatable — I can add new NCERT books without retraining anything.
- Why ChromaDB?:** Runs locally, no cloud account needed, production-ready. For scale I would move to Pinecone or Weaviate — same LangChain interface, just different backend.
- Why Groq?:** Free tier, fastest LLM inference available, runs Llama 3 which is strong for educational Q&A.; Easily swappable with OpenAI or Anthropic API.
- How do you prevent hallucination?:** The prompt strictly instructs the LLM to only use the provided context. If no relevant chunks are found, it returns a 'I don't have information on this' response.
- Can this scale to production?:** Yes — replace ChromaDB with Pinecone, Groq with Azure OpenAI, Streamlit with FastAPI backend. Same RAG pipeline, different infrastructure.

## 12. SKILLS THIS PROJECT ADDS TO YOUR RESUME

After building this project your GenAI fit score jumps from 62% to 90%+. These are the skills you can now honestly add to your resume and Wellfound profile:

| Skill        | Your Evidence                                                                    | ATS Keyword Value                               |
|--------------|----------------------------------------------------------------------------------|-------------------------------------------------|
| LangChain    | Built full RAG chain using LCEL — document loader, text splitter, retriever, LLM | VERY HIGH — in 90% of GenAI job postings        |
| RAG Pipeline | Core architecture — ingest, embed, store, retrieve, generate                     | VERY HIGH — explicitly listed in most GenAI JDs |

| Skill                              | Your Evidence                                             | ATS Keyword Value                                     |
|------------------------------------|-----------------------------------------------------------|-------------------------------------------------------|
| Vector Database / ChromaDB         | Ingested NCERT PDFs, stored and queried embeddings        | VERY HIGH — Pinecone/Weaviate/ChromaDB all equivalent |
| Embeddings / sentence-transformers | Used all-MiniLM-L6-v2 for semantic text embedding         | HIGH — shows ML depth beyond API calls                |
| Groq API                           | LLM generation step — already in ZENI, now in RAG context | HIGH — fast inference, Llama 3 experience             |
| Prompt Engineering                 | Designed context-aware prompts with language switching    | HIGH — in every GenAI posting                         |
| Streamlit Deployment               | Live URL on Streamlit Cloud with public access            | MEDIUM — proof of deployment experience               |
| Document Processing (pypdf)        | Loaded, extracted and chunked real-world PDF documents    | MEDIUM — practical data engineering skill             |

## 13. YOUR DAILY SCHEDULE DURING THE BUILD

| Time               | Task                                                                                                     |
|--------------------|----------------------------------------------------------------------------------------------------------|
| 9:00 AM – 12:00 PM | Apply for CV + GenAI jobs on Wellfound, Instahyre, Cutshort. Send 5–10 targeted applications per day.    |
| 12:00 PM – 1:00 PM | Lunch + read one LangChain or RAG concept page (20 min). No coding during this time.                     |
| 1:00 PM – 6:00 PM  | Build the project — one phase per day. Use Claude or ChatGPT as your coding agent. Test after each step. |
| 6:00 PM – 7:00 PM  | Push code to GitHub. Write a meaningful commit message. Update README with progress.                     |
| 7:00 PM – 8:00 PM  | Review what you built. Plan the next day's phase. Test edge cases. Fix anything broken.                  |

## 14. DONE CHECKLIST — PROJECT IS COMPLETE WHEN ALL 11 ARE CHECKED

- 1. ingest.py runs without errors and prints the number of chunks created
- 2. chroma\_db/ folder is created and contains vector data
- 3. rag\_pipeline.py returns a correct NCERT-based answer to a test question in Python shell
- 4. Streamlit UI shows question input, language selector, and answer
- 5. Hindi mode returns a Hindi response from Groq without any translation API
- 6. Source document names are displayed below each answer in the UI
- 7. Project is pushed to GitHub with .env in .gitignore
- 8. README.md is written with project description and tech stack
- 9. Deployed on Streamlit Cloud with a live public URL
- 10. Wellfound profile updated with LangChain, RAG, ChromaDB, Vector Database skills
- 11. You can explain the full architecture verbally in 2 minutes without notes

When all 11 boxes are checked — your GenAI fit goes from 62% to 90%+. You have a live deployed product, real LangChain + RAG experience, and a credible answer to every GenAI interview question. You are ready to apply. Start building.